



UNIVERSITE D'ANTANANARIVO

ECOLE SUPERIEURE POLYTECHNIQUE

MENTION TELECOMMUNICATION



Domaine : Sciences de l'Ingénieur

Mention : Télécommunication

Rapport Application Mobile

Par :

RAFANOMEZANTSOA Sitraka Elysa

AMBININTSOA Hasina Julio

RANDRIANARIVONY Irinasoa Nantenaina

TANTELY RABEARIVELO Miarantsoa

Enseignant :

Mr Mirado

CHAPITRE 1 APPLICATION 1: EVENEMENTS GET

1.1 Présentation générale du projet

GET Events est une application mobile destinée aux étudiants et aux enseignants de la mention Télécommunication, permettant de consulter les événements organisés (galas, conférences, soirées, concours, etc.), d'acheter des billets en ligne via mobile money et de récupérer un billet électronique sous forme de QR code. Le projet est composé de deux briques principales : une application Android développée en Kotlin avec Jetpack Compose pour les utilisateurs (étudiants) et les administrateurs, et une API backend développée avec le framework Ktor en Kotlin, connectée à une base de données relationnelle.

Le système met en relation trois types d'acteurs : l'étudiant qui consulte les événements et achète ses billets, le personnel d'administration qui valide les inscriptions, les paiements et gère le contenu, et le système lui-même qui assure l'authentification, la génération des billets et l'envoi de notifications.

1.1.1 Objectifs

- Centraliser la communication autour des événements organisés au sein de l'établissement.
- Permettre l'inscription et l'authentification sécurisée des étudiants, avec validation par un administrateur (vérification de la carte d'étudiant).
- Digitaliser la vente de billets et le paiement via mobile money (Orange Money, Mvola).
- Générer automatiquement un billet électronique sous forme de QR code, scannable à l'entrée de l'événement.
- Gérer les inscriptions à des concours spécifiques (ex : compétitions par équipe).
- Fournir un espace d'administration complet pour la gestion des événements, des commandes, des paiements, des concours et des comptes utilisateurs.
- Informer les utilisateurs en temps réel via un système de notifications internes.

1.1.2 Acteurs et profils utilisateurs

Acteur	Rôle dans l'application
Étudiant (USER)	S'inscrit, consulte les événements, achète des billets, s'inscrit aux concours, reçoit des notifications et consulte ses billets électroniques (QR codes).

Administrateur (ADMIN)	Gère les événements, valide les inscriptions des étudiants (carte d'étudiant), valide ou annule les paiements, gère les concours et supervise l'activité globale.
Système / API	Authentifie les utilisateurs, applique les règles métier (quotas de billets, statuts de commande), génère les QR codes et envoie les notifications.

1.2 Architecture technique

Le projet adopte une architecture client-serveur classique, organisée en deux sous-projets distincts au sein d'un même dépôt (mono-repo Gradle) : un module « frontend » (application Android) et un module « backend » (API REST Ktor), partageant un système de build Gradle commun.

1.2.1 Vue d'ensemble

- Frontend (mobile) : application Android native, développée en Kotlin, avec une interface utilisateur déclarative basée sur Jetpack Compose et Material 3.
- Backend (API) : serveur Ktor exposant une API REST en JSON, organisée en couches Repository / Service / Routing.
- Base de données : H2 (fichier local, utilisée par défaut en développement) ou MySQL 8 (utilisée en production via Docker).
- Stockage de fichiers : un répertoire « uploads » contenant les QR codes générés (uploads/qrcodes) et les photos des cartes d'étudiant (uploads/student-cards), exposés en fichiers statiques par le serveur.
- Conteneurisation : un fichier docker-compose.yml orchestre la base MySQL et le service backend (avec persistance des données et des fichiers uploadés via des volumes Docker).

1.2.2 Backend — API REST (Ktor)

Le backend est structuré selon une architecture en couches, séparant clairement les responsabilités :

- Models : définitions des entités métier (Event, Order, User, Notification, ContestRegistration, PasswordResetToken) et des tables Exposed correspondantes (mapping objet-relationnel).
- Repositories : couche d'accès aux données, encapsulant les requêtes SQL via le framework Exposed (UserRepository, EventRepository, OrderRepository, NotificationRepository, ContestRegistrationRepository, PasswordResetRepository).
- Services : logique métier (AuthService, EventService, OrderService, ContestService, NotificationService) — validation des règles, calculs, orchestration entre repositories.

- Routing : définition des routes HTTP exposées par l'API et liaison avec les services.
- Configuration : fichiers Security.kt (authentification JWT), StatusPages.kt (gestion des erreurs), Serialization.kt (JSON), Exposed.kt (connexion base de données) et DatabaseSeed.kt (jeu de données de démonstration au premier démarrage).

1.2.3 Frontend — Application Android (Jetpack Compose)

L'application mobile suit une architecture orientée MVVM (Model-View-ViewModel), organisée comme suit :

- ui/screens : les écrans de l'application, répartis entre le parcours utilisateur (authentification, accueil, détail d'événement, paiement, profil) et le parcours administrateur (tableau de bord, gestion des événements, des inscriptions, des paiements et des concours).
- ui/components et ui/theme : composants réutilisables et thème graphique (couleurs, typographies Material 3).
- navigation : graphes de navigation Compose (Navigation.kt, Routes.kt et un sous-graphe dédié à l'administration : AdminNavGraph.kt, AdminRoutes.kt).
- data : modèles de données côté client, interfaces Retrofit pour la communication avec l'API (data/api), repositories (data/repository) et stockage local des préférences (data/local, via DataStore).
- viewmodel : gestion d'état et logique de présentation pour chaque écran, exposée aux composables via des flux d'état (StateFlow).

1.3 Technologies et bibliothèques utilisées

1.3.1 Backend

Composant	Technologie / Bibliothèque
Langage	Kotlin
Framework serveur	Ktor (Netty, Content Negotiation, Resources, CORS, Status Pages, Call Logging)
Accès aux données	Exposed (Core, DAO, JDBC) — ORM Kotlin
Pool de connexions	HikariCP
Base de données	H2 (développement) / MySQL 8 (production)

Authentification	JWT (ktor-server-auth-jwt)
Hachage des mots de passe	bcrypt (at.favre.lib:bcrypt)
Génération de QR codes	ZXing (com.google.zxing core / javase)
Envoi d'e-mails	Jakarta Mail (réinitialisation de mot de passe)
Journalisation	Logback
Tests	Kotlin Test, Ktor Server Test Host, H2

1.3.2 Frontend (Android)

Composant	Technologie / Bibliothèque
Langage	Kotlin
Interface utilisateur	Jetpack Compose + Material 3 (Compose BOM)
Navigation	AndroidX Navigation Compose
Architecture	Lifecycle ViewModel Compose (MVVM)
Appels réseau	Retrofit + convertisseur Gson, OkHttp Logging Interceptor
Asynchronisme	Kotlin Coroutines
Stockage local	AndroidX DataStore Preferences (session / token)
Chargement d'images	Coil Compose

QR codes	ZXing Core (génération / lecture côté client)
Tests	JUnit, AndroidX Espresso

1.3.3 Infrastructure et déploiement

- Conteneurisation via Docker (Dockerfile dédié au backend) et orchestration avec Docker Compose.
- Base de données MySQL 8.0 initialisée via un script SQL (scripts/init-mysql.sql), avec un volume Docker persistant.
- Persistance des fichiers uploadés (QR codes, cartes d'étudiant) via un volume Docker dédié (uploads_data).
- Configuration entièrement pilotée par variables d'environnement (DB_MODE, DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET, PORT), facilitant le passage du mode développement (H2) au mode production (MySQL).

1.4 Fonctionnalités principales

1.4.1 Authentification et gestion des comptes

Le module d'authentification gère l'ensemble du cycle de vie d'un compte utilisateur :

- Inscription en plusieurs étapes (RegisterStep1, RegisterStep2, RegisterStep3) collectant les informations personnelles, le type d'utilisateur (étudiant, enseignant...), la mention et l'année d'étude, ainsi qu'une photo de la carte d'étudiant.
- Connexion sécurisée par e-mail et mot de passe, avec génération d'un jeton JWT (JSON Web Token) contenant l'identifiant, l'e-mail et le rôle de l'utilisateur.
- Validation manuelle des inscriptions par un administrateur (statut PENDING / approuvé), garantissant que seuls les étudiants reconnus accèdent aux fonctionnalités d'achat.
- Récupération de mot de passe par e-mail (mot de passe oublié / réinitialisation), avec jetons de réinitialisation à durée de vie limitée.
- Gestion du profil utilisateur (consultation et mise à jour des informations personnelles).

1.4.2 Gestion des événements

- Consultation de la liste des événements avec recherche et pagination.
- Consultation du détail d'un événement : titre, description, date, lieu, image, tarifs (étudiant et enseignant) et nombre de billets restants.
- Côté administrateur : création, modification et suppression des événements (titre, description, date, lieu, tarifs, quota total de billets, image).
- Suivi en temps réel du nombre de billets restants par événement, mis à jour automatiquement à chaque commande validée.

1.4.3 Billetterie et paiement mobile

La billetterie constitue le cœur fonctionnel de l'application :

- Choix du type de billet (étudiant ou enseignant) et de la quantité souhaitée.
- Paiement via mobile money : Orange Money ou Mvola, avec saisie d'une référence de transaction.
- Création d'une commande à l'état « En attente » (PENDING) dans l'attente de la validation du paiement par un administrateur.
- Une fois la commande validée (statut VALIDATED), un billet électronique est généré automatiquement sous forme de QR code (bibliothèque ZXing), stocké dans uploads/qrcodes et accessible via une URL statique.
- Suivi du statut de la commande par l'utilisateur (en attente, validée, annulée, utilisée — le statut USED correspondant au billet scanné à l'entrée de l'événement).
- Historique complet des commandes de l'utilisateur, avec détail de chaque commande.

1.4.4 Concours et inscriptions par équipe

- Inscription des étudiants à un concours associé à un événement, via un formulaire dédié (ContestRegisterScreen).
- Suivi du nombre d'équipes inscrites par événement (compteur dédié).
- Validation ou rejet des inscriptions au concours par un administrateur, depuis un écran dédié.

1.4.5 Notifications

- Système de notifications internes informant l'utilisateur des évolutions importantes (validation de commande, validation d'inscription, etc.).
- Compteur de notifications non lues, affiché dans l'interface.
- Marquage d'une notification (ou de l'ensemble des notifications) comme lue.

1.4.6 Espace administrateur

L'application embarque un espace d'administration accessible aux comptes disposant du rôle « ADMIN », avec authentification dédiée :

- Tableau de bord (DashboardAdminScreen) offrant une vue synthétique de l'activité.
- Gestion des événements (création, édition, suppression).
- Gestion des inscriptions utilisateurs : vue d'ensemble, détail d'un inscrit, approbation ou rejet des comptes en attente de validation (vérification de la carte étudiant).
- Gestion des paiements : liste des commandes, validation ou annulation d'un paiement, ce qui déclenche la génération du billet QR code correspondant.
- Gestion des concours : consultation et validation/rejet des inscriptions aux concours.

1.5 Modèle de données

La base de données relationnelle (H2 en développement, MySQL en production) repose sur six entités principales, reliées entre elles par des clés étrangères logiques (gérées au niveau applicatif via le framework Exposed) :

Entité	Description
Users	Comptes utilisateurs : identité, rôle (USER / ADMIN), statut d'approbation, type d'utilisateur, mention, année d'étude, carte d'étudiant, statut d'inscription.
Events	Événements : titre, description, date, lieu, tarifs étudiant/enseignant, quota total et billets restants, image, créateur.
Orders	Commandes de billets : utilisateur, événement, quantité, montant total, statut (PENDING / VALIDATED / CANCELLED / USED), méthode et référence de paiement, URL du QR code.
ContestRegistrations	Inscriptions aux concours liés à un événement, avec statut de validation.
Notifications	Notifications adressées aux utilisateurs, avec indicateur de lecture.
PasswordResetTokens	Jetons temporaires utilisés pour la réinitialisation de mot de passe.

Le statut d'une commande (OrderStatus) constitue le pivot du cycle de vie d'un billet, avec quatre états possibles : PENDING (commande créée, en attente de validation du paiement), VALIDATED (paiement validé, billet QR code généré), CANCELLED (commande annulée) et USED (billet scanné et utilisé à l'entrée de l'événement).

1.6 API REST exposée par le backend

L'ensemble des échanges entre l'application mobile et le serveur s'effectue au format JSON via une API REST. Les routes sont regroupées en quatre familles : routes publiques, routes d'authentification, routes utilisateur (protégées par JWT) et routes d'administration (protégées par JWT + rôle ADMIN).

1.6.1 Routes publiques et techniques

Route	Description
GET /	Point d'entrée de vérification (message de statut de l'API).
GET /health	Contrôle de santé du service.
GET /test-db	Vérification de la connexion à la base de données.
GET /events	Liste des événements (recherche, pagination).
GET /events/{id}	Détail d'un événement.
GET /events/{id}/contest-team-count	Nombre d'équipes inscrites au concours d'un événement.
/qrcores/* et /student-cards/*	Fichiers statiques : QR codes des billets et photos des cartes d'étudiant.

1.6.2 Authentification

Route	Description
POST /auth/register	Inscription d'un nouvel utilisateur.
POST /auth/login	Connexion et émission d'un jeton JWT.
POST /auth/forgot-password	Demande de réinitialisation de mot de passe (envoi d'un e-mail).

POST /auth/reset-password	Réinitialisation effective du mot de passe via jeton.
---------------------------	---

1.6.3 Espace utilisateur (authentifié)

Route	Description
POST /user/orders	Création d'une commande de billets.
GET /user/orders	Historique des commandes de l'utilisateur.
GET /user/orders/{orderId}	Détail d'une commande (billet, QR code).
POST /user/contest-registrations	Inscription à un concours.
GET /user/profile	Consultation du profil.
PUT /user/profile	Mise à jour du profil.
GET /user/notifications	Liste des notifications.
GET /user/notifications/unread-count	Nombre de notifications non lues.
PUT /user/notifications/{id}/read	Marquer une notification comme lue.
PUT /user/notifications/read-all	Marquer toutes les notifications comme lues.

1.6.4 Espace administrateur (rôle ADMIN requis)

Route	Description
POST /admin/events	Création d'un événement.
PUT /admin/events/{id}	Modification d'un événement.
DELETE /admin/events/{id}	Suppression d'un événement.
GET /admin/orders	Liste de toutes les commandes.
PUT /admin/orders/{orderId}/validate	Validation d'un paiement et génération du billet QR code.
PUT /admin/orders/{orderId}/cancel	Annulation d'une commande.
GET /admin/users	Liste des comptes utilisateurs.
PUT /admin/users/{userId}/approve	Approbation d'une inscription utilisateur.
PUT /admin/users/{userId}/reject	Rejet d'une inscription utilisateur.
GET /admin/contest-registrations	Liste des inscriptions aux concours.
PUT /admin/contest-registrations/{id}/approve	Validation d'une inscription au concours.

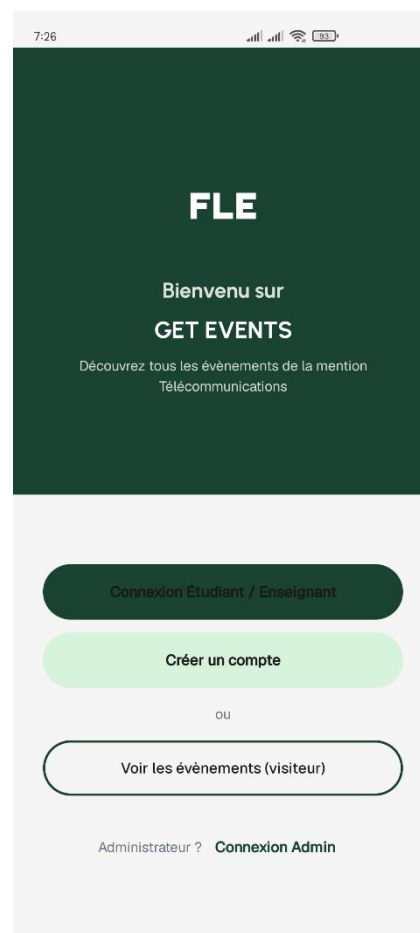
PUT /admin/contest- registrations/{id}/reject	Rejet d'une inscription au concours.
---	--------------------------------------

1.7 Sécurité

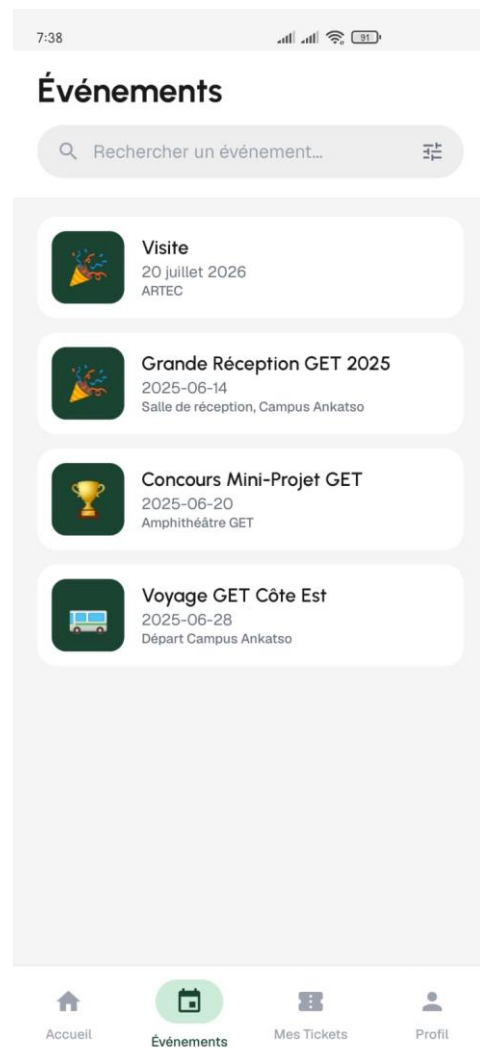
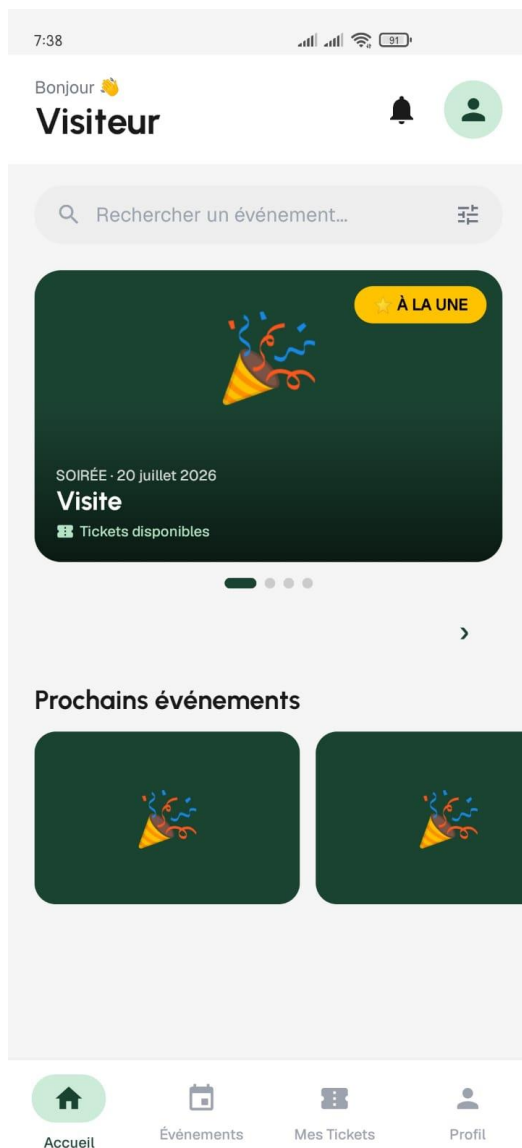
- Authentification basée sur des jetons JWT (signature HMAC256), portant l'identifiant de l'utilisateur, son e-mail et son rôle.
- Mots de passe stockés sous forme de hachages bcrypt, jamais en clair.
- Contrôle d'accès par rôle : les routes d'administration vérifient explicitement que l'utilisateur authentifié possède le rôle ADMIN.
- Validation des inscriptions par un administrateur avant qu'un compte étudiant ne puisse effectuer des achats (lutte contre les comptes frauduleux).
- Réinitialisation de mot de passe sécurisée par jeton à usage unique et à durée de vie limitée, envoyé par e-mail.
- Gestion centralisée des erreurs HTTP via Status Pages, évitant la fuite d'informations techniques sensibles.

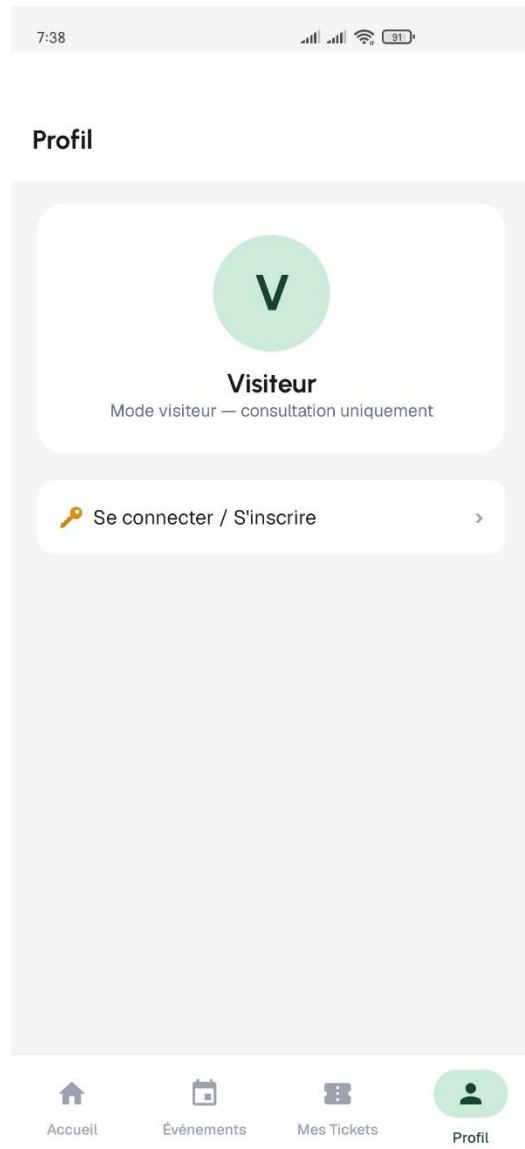
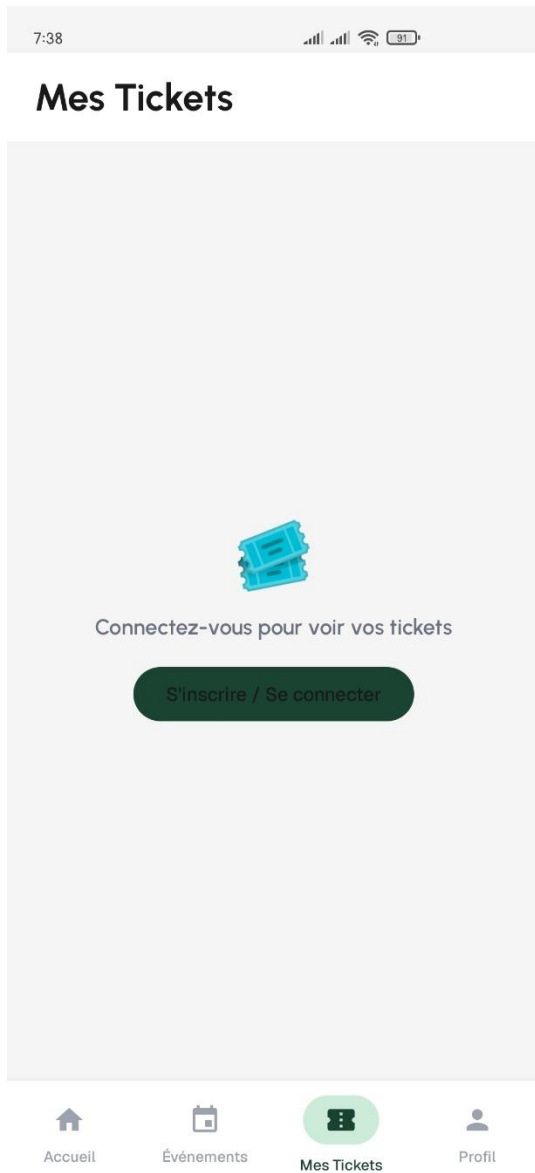
1.8 Présentation des écrans

1.8.1 SplashScreen et écran d'accueil



1.8.2 Ecrans visiteur





1.8.3 Création de compte utilisateur (étudiant ou enseignant)

The figure displays four screenshots of a mobile application interface, arranged in a 2x2 grid, showing the 'Créer un compte' (Create account) process. Each screenshot shows a progress bar at the top with three steps: 1 (checked), 2 (current), and 3. The title 'Créer un compte' is at the top left of each screen.

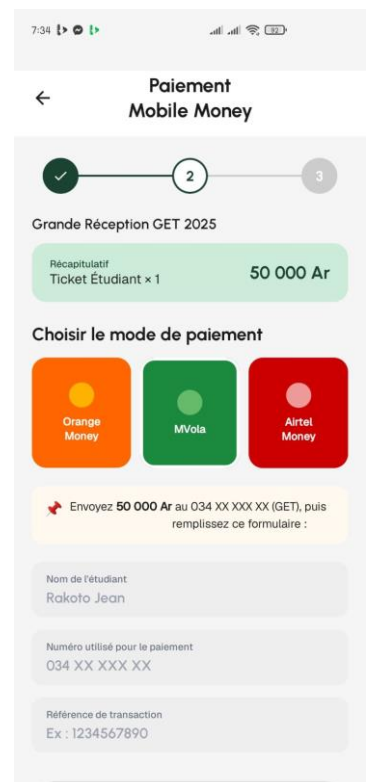
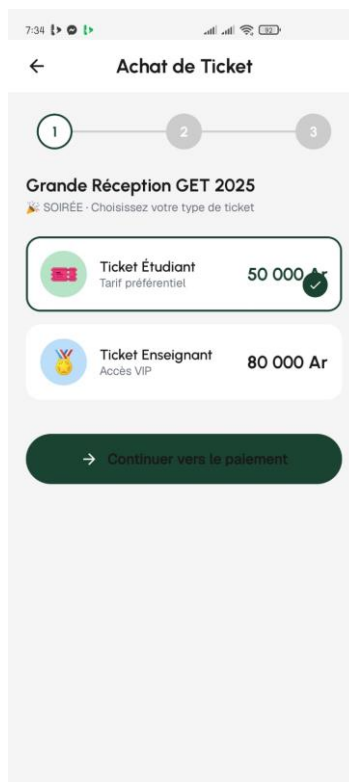
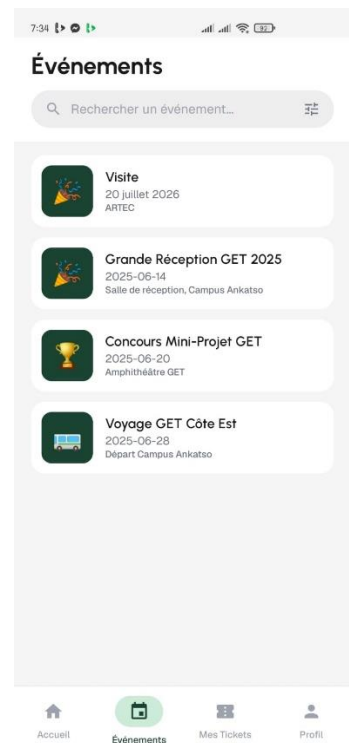
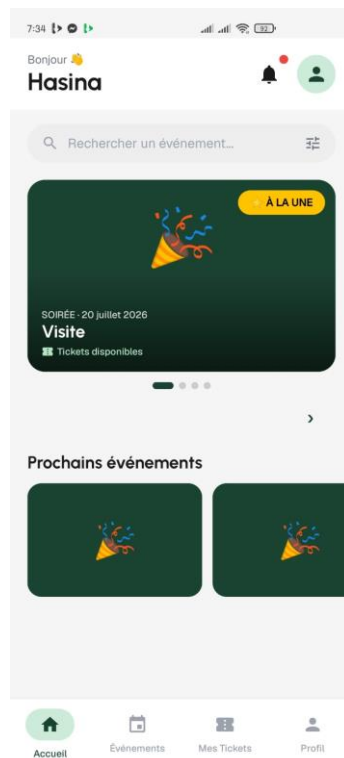
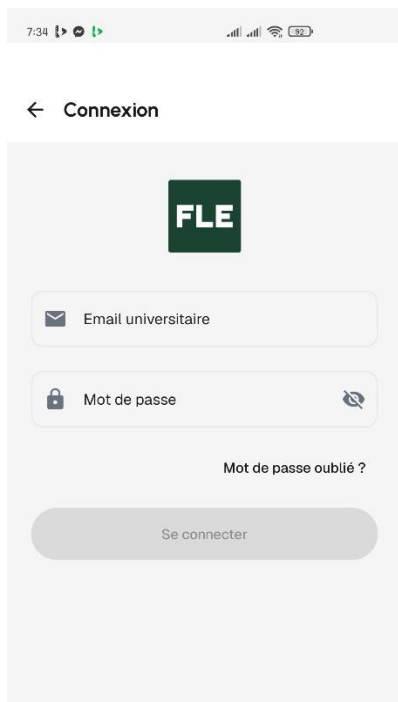
Top Left Screenshot (Instructor): The progress bar shows step 2 is active. The title is 'Créer un compte'. The screen is titled 'Étape 2 — Informations universitaires'. It features a 'Type de compte' dropdown menu with 'Enseignant' selected. Below it is a 'Mention' field with a graduation cap icon and the text 'Télécommunications'. At the bottom is a dark green button labeled 'Suivant →'.

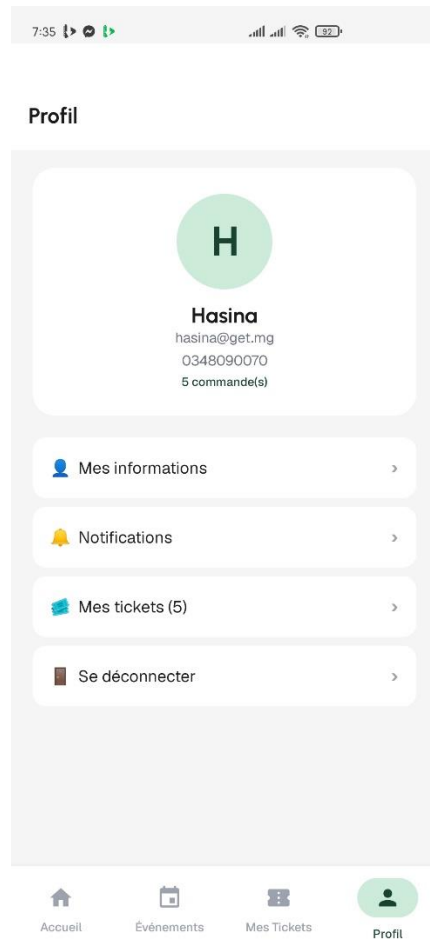
Top Right Screenshot (Student): The progress bar shows step 2 is active. The title is 'Créer un compte'. The screen is titled 'Étape 2 — Informations universitaires'. It features a 'Type de compte' dropdown menu with 'Étudiant' selected. Below it is a 'Mention' field with a graduation cap icon and the text 'Télécommunications'. Below that is an 'Année / Niveau' dropdown menu with 'M1' selected. At the bottom is a light gray button labeled 'Suivant →'.

Bottom Left Screenshot (Instructor): The progress bar shows step 2 is active. The title is 'Créer un compte'. The screen is titled 'Étape 2 — Informations universitaires'. It features a 'Type de compte' dropdown menu with 'Enseignant' selected. Below it is a 'Mention' field with a graduation cap icon and the text 'Télécommunications'. At the bottom is a dark green button labeled 'Suivant →'.

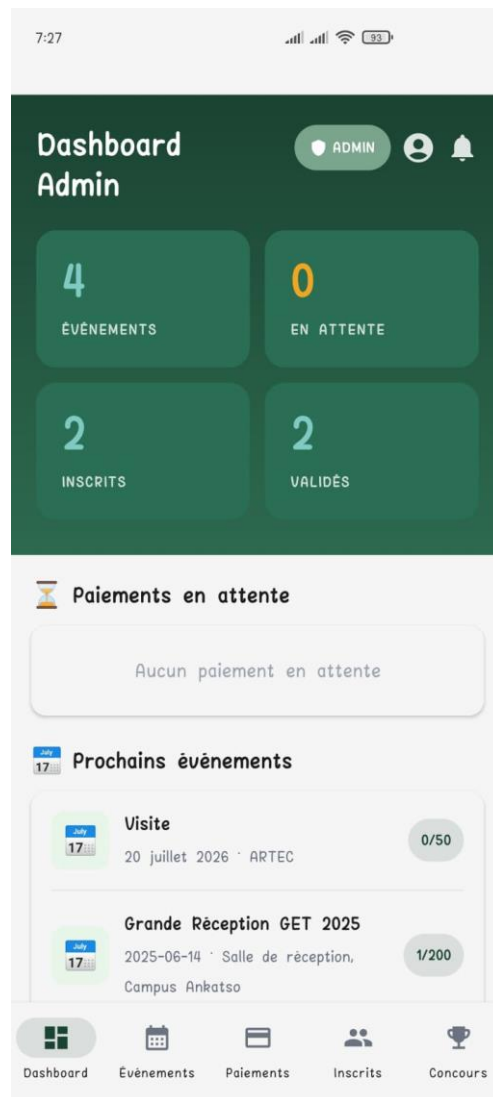
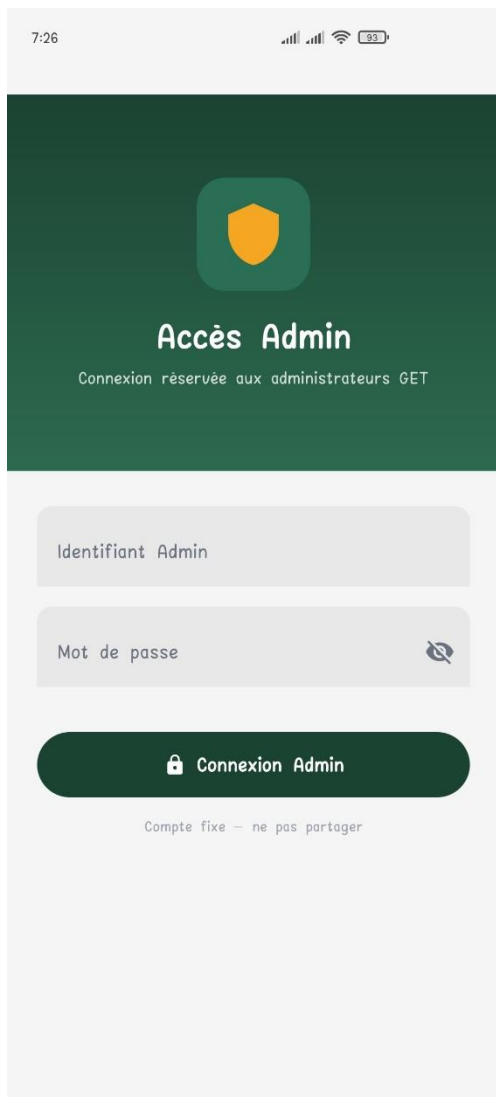
Bottom Right Screenshot (Student): The progress bar shows step 2 is active. The title is 'Créer un compte'. The screen is titled 'Étape 2 — Informations universitaires'. It features a 'Type de compte' dropdown menu with 'Étudiant' selected. Below it is a 'Mention' field with a graduation cap icon and the text 'Télécommunications'. Below that is an 'Année / Niveau' dropdown menu with 'M1' selected. At the bottom is a dark green button labeled 'Suivant →'.

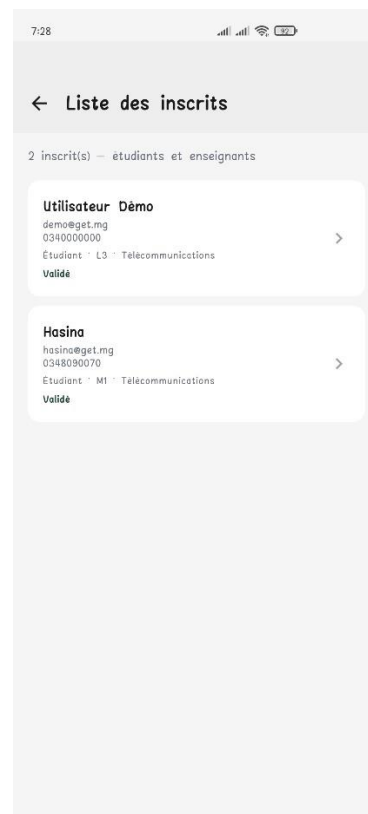
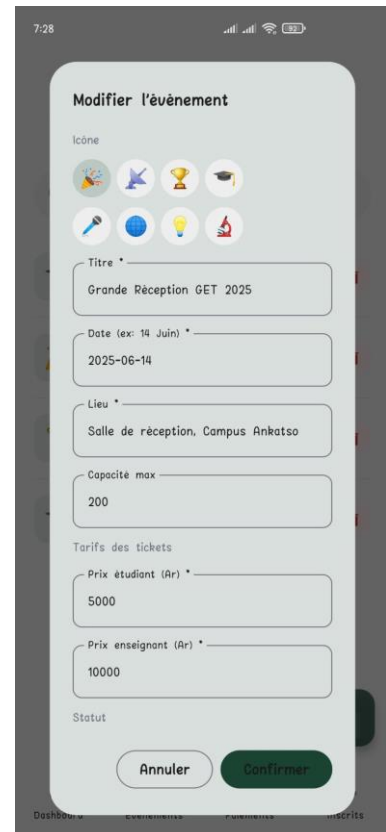
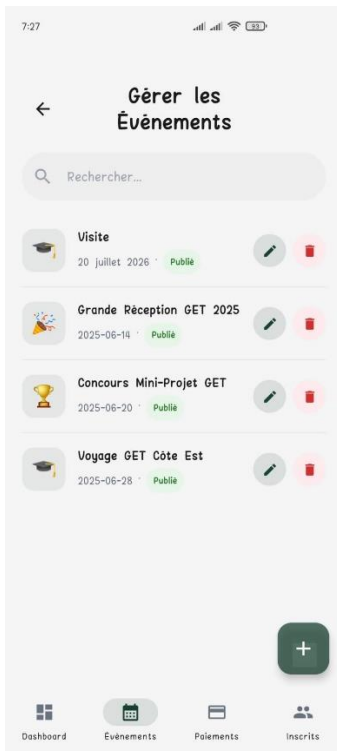
1.8.4 Ecrans utilisateur



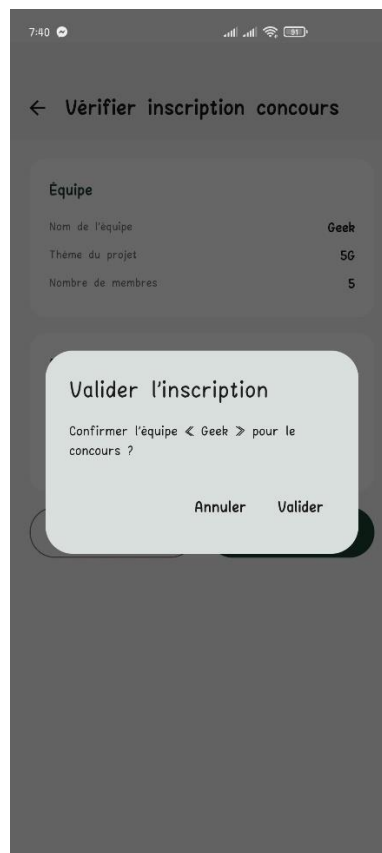
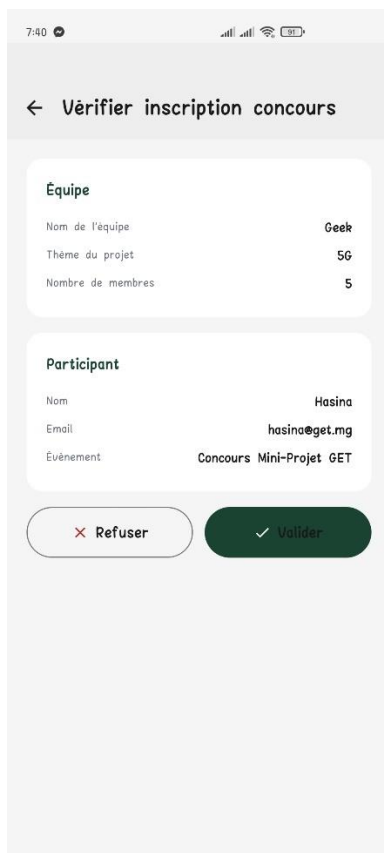
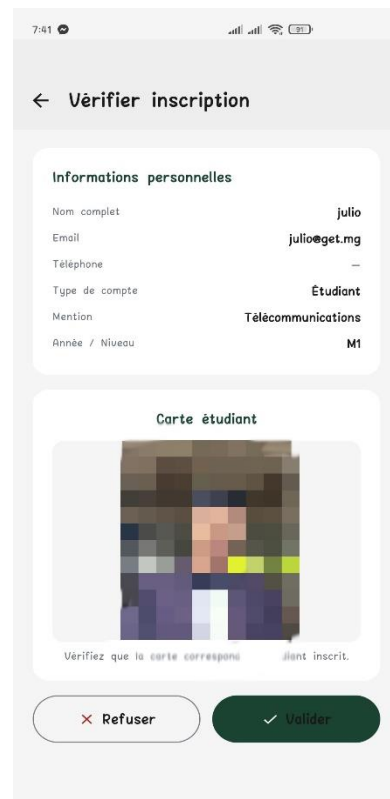
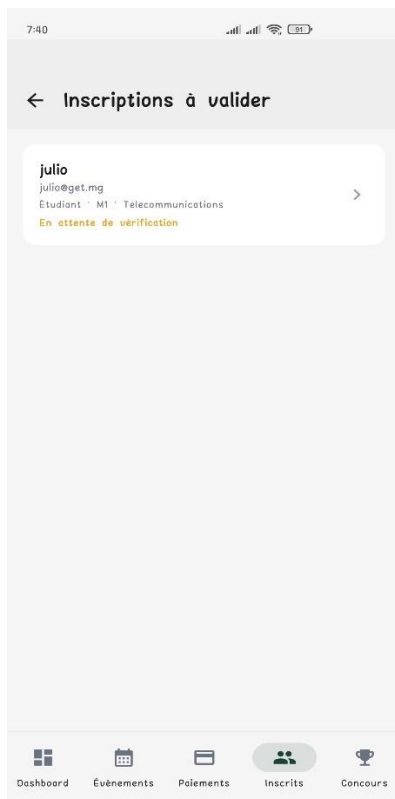


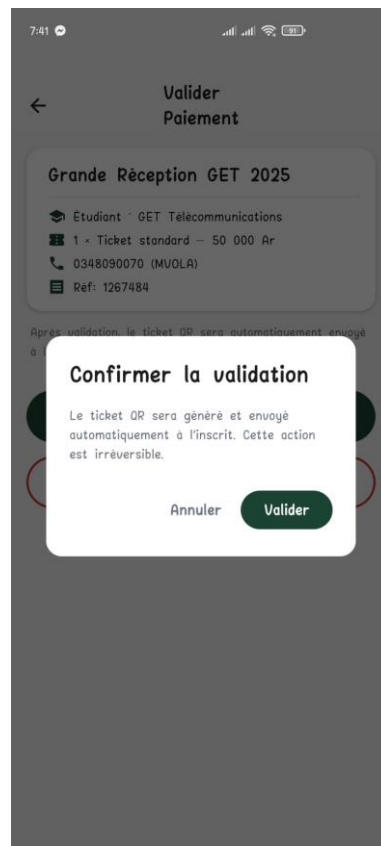
1.8.5 Ecran Admin





1.8.6 Validation Admin





1.8.7 Mot de passe oublié

7:39

← Nouveau mot de passe

Un code a été envoyé à
hasina@get.mg

123 Code de vérification

Nouveau mot de passe

Confirmer le mot de passe

Réinitialiser le mot de passe

7:39

← Mot de passe oublié

Entrez votre email universitaire. Nous vous enverrons un
code de vérification pour réinitialiser votre mot de passe.

Email universitaire

Envoyer le code

Retour à la connexion

CHAPITRE 2 Application de scan de tickets

2.1 Présentation générale du projet

GET Ticket est une application mobile complémentaire au projet GET Events. Alors que GET Events permet aux étudiants de découvrir des événements et d'acheter leurs billets, GET Ticket est l'outil destiné au personnel d'accueil (agents de contrôle) chargé de scanner les billets électroniques (QR codes) à l'entrée des événements afin de valider, refuser ou détecter les billets déjà utilisés.

L'application se présente sous la forme d'un scanner mobile temps réel : l'agent se connecte avec un compte administrateur, l'application active la caméra du téléphone, détecte automatiquement un QR code, l'envoie au serveur pour vérification, puis affiche immédiatement le résultat (ticket valide, refusé ou déjà utilisé) avec les informations du porteur du billet.

Un point essentiel de l'architecture est le partage de la base de données : GET Ticket se connecte à la même base de données (« getevents_db ») que l'application GET Events, et n'ajoute qu'une seule table supplémentaire (scan_history) destinée à l'historique des scans, sans modifier le

schéma existant. Toute validation effectuée dans GET Ticket met donc directement à jour le statut des commandes consultables depuis GET Events.

2.1.1 Objectifs

- Fournir aux agents de contrôle un outil simple et rapide pour vérifier la validité des billets à l'entrée d'un événement.
- Détecter automatiquement les tentatives de fraude : billets inexistants, déjà utilisés (doublons), annulés ou en attente de paiement.
- Marquer automatiquement un billet valide comme « utilisé » (statut USED) dès son premier scan réussi, empêchant toute réutilisation.
- Afficher en temps réel des statistiques de contrôle (billets validés, restants, refusés).
- Conserver un historique consultable et filtrable de tous les scans effectués, avec leur statut et le motif de refus éventuel.
- Réutiliser l'infrastructure existante (base de données, comptes administrateurs) du projet GET Events, sans duplication de données.

2.1.2 Acteurs et profils utilisateurs

Acteur	Rôle dans l'application
Agent de contrôle (ADMIN)	Se connecte avec un compte administrateur GET Events, scanne les QR codes des billets à l'entrée de l'événement, consulte l'historique et les statistiques de contrôle.
Système / API GET Ticket	Authentifie l'agent, interprète le contenu du QR code, vérifie le statut de la commande correspondante dans la base partagée, met à jour ce statut et enregistre chaque scan dans l'historique.
Base de données partagée	Source unique de vérité pour les utilisateurs et les commandes, alimentée par GET Events et consultée/modifiée par GET Ticket.

2.2 Architecture technique

Comme GET Events, le projet GET Ticket est structuré en deux modules : une application Android (« app ») et une API backend Ktor (« backend »), regroupés dans un même dépôt Gradle. Sa particularité est de fonctionner comme un second service applicatif venant compléter l'écosystème GET Events, en partageant la même base de données relationnelle.

2.2.1 Vue d'ensemble

- Frontend (mobile) : application Android native en Kotlin avec Jetpack Compose et Material 3, intégrant la capture vidéo en temps réel (CameraX) et la reconnaissance de QR codes (Google ML Kit Barcode Scanning).
- Backend (API) : second serveur Ktor, exposé par défaut sur le port 8081 (alors que GET Events utilise le port 8080), structuré en trois fichiers principaux : Main.kt (configuration de l'application), Routes.kt (logique métier et endpoints) et Database.kt (connexion et tables).
- Base de données partagée : connexion à la même base « getevents_db » (H2 en local ou MySQL en production) que GET Events. GET Ticket lit les tables Users et Orders définies par GET Events, et crée une table additionnelle ScanHistory dédiée à son propre historique de scans.
- Authentification : système JWT indépendant mais compatible — basé sur le même secret applicatif, GET Ticket vérifie que l'utilisateur connecté possède le rôle ADMIN avant d'autoriser l'accès au scanner.
- Déploiement : le docker-compose.yml du projet GET Events référence directement ce second backend (service « backend2 » / « getticket_backend »), exposé sur le port 8081 et connecté au même conteneur MySQL.

2.2.2 Backend — API REST (Ktor)

Le backend GET Ticket est volontairement minimaliste et se concentre sur la logique de vérification de billets :

- Main.kt : point d'entrée du serveur, installation des plugins Ktor (sérialisation JSON, CORS, gestion centralisée des erreurs via Status Pages, authentification JWT) et déclaration des routes.
- Database.kt : connexion à la base partagée via HikariCP (pool de connexions), avec bascule H2/MySQL pilotée par variables d'environnement ; définition des tables Users et Orders (en miroir de GET Events, lecture seule) et de la nouvelle table ScanHistory (créée automatiquement si absente).
- Routes.kt : implémentation de la connexion, de la récupération de l'événement actif, des statistiques, de la vérification/validation de billets et de l'historique des scans.
- PasswordUtil.kt : vérification des mots de passe hachés (bcrypt), compatible avec les comptes créés par GET Events.

2.2.3 Frontend — Application Android (Jetpack Compose)

L'application suit une architecture simple à base d'état Compose, organisée en cinq écrans principaux gérés par une machine à états (sealed class Ecran) au sein de MainActivity :

- EcranConnexion : authentification de l'agent (e-mail + mot de passe) auprès de l'API GET Ticket.
- EcranScanner : écran principal, affichant le flux de la caméra via CameraX, détectant les QR codes avec ML Kit et envoyant le code détecté au serveur pour vérification ; affiche également les statistiques en temps réel et donne accès à l'historique.

- `EcranResultatValide` : écran de confirmation affiché lorsque le billet est valide, avec les informations du porteur (nom, catégorie, promotion, code et type de ticket), puis retour automatique au scanner après quelques secondes.
- `EcranResultatRefuse` : écran affiché lorsque le billet est refusé (déjà utilisé, annulé, en attente de paiement ou inconnu), avec le motif précis du refus.
- `EcranHistorique` : liste consultable et filtrable (recherche par nom) de l'ensemble des scans effectués, avec leur statut (valide / refusé / doublon).
- `data/api/ApiClient` : client HTTP basé sur Ktor Client (moteur Android), assurant la communication JSON avec le backend et reproduisant les modèles de données du serveur.
- `ui/theme` : thème graphique dédié (palette de verts — `GreenDark`, `GreenMedium`, `GreenLight`, `GreenMint` — et couleur d'alerte `BadgeRed`), distinct de celui de GET Events.

2.3 Technologies et bibliothèques utilisées

2.3.1 Backend

Composant	Technologie / Bibliothèque
Langage	Kotlin
Framework serveur	Ktor 2.3.7 (Netty, Content Negotiation, CORS, Status Pages, Auth JWT)
Accès aux données	Exposed 0.45.0 (Core, DAO, JDBC) — ORM Kotlin
Pool de connexions	HikariCP 6.0.0
Base de données	H2 (développement, base partagée avec GET Events) / MySQL 8 (production)
Authentification	JWT (Auth0 java-jwt 4.4.0)
Hachage des mots de passe	bcrypt (at.favre.lib:bcrypt) — compatible avec GET Events
Journalisation	Logback

2.3.2 Frontend (Android)

Composant	Technologie / Bibliothèque
Langage	Kotlin
Interface utilisateur	Jetpack Compose 1.6.1 + Material 3 (+ icônes étendues)
Appels réseau	Ktor Client (moteur Android) avec Content Negotiation JSON
Capture vidéo temps réel	CameraX (camera2, lifecycle, view) 1.3.1
Lecture de QR codes	Google ML Kit — Barcode Scanning 17.2.0
Gestion des permissions	Accompanist Permissions 0.34.0 (accès caméra)
Asynchronisme	Kotlin Coroutines

2.3.3 Infrastructure et déploiement

- Service backend conteneurisé (« backend2 » / « getticket_backend ») au sein du même docker-compose.yml que GET Events, exposé sur le port 8081.
- Connexion au même conteneur MySQL (« mysql ») et à la même base « getevents_db » que GET Events, garantissant la cohérence des données entre les deux applications.
- Configuration pilotée par variables d'environnement (DB_MODE, DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET, PORT), avec un secret JWT identique à celui de GET Events pour assurer la compatibilité des comptes administrateurs.
- En mode développement local (H2), GET Ticket se connecte directement au fichier de base de données généré par GET Events (chemin relatif vers Application 1/data/getevents_db).

2.4 Fonctionnalités principales

2.4.1 Connexion (espace réservé aux administrateurs)

- Authentification par e-mail et mot de passe, vérifiée contre la table des utilisateurs de la base partagée.
- Accès strictement réservé aux comptes possédant le rôle ADMIN dans GET Events ; tout autre compte est rejeté (statut HTTP 403).

- En cas de succès, émission d'un jeton JWT (valable 24 heures) contenant l'identifiant, l'e-mail et le rôle de l'agent, utilisé pour authentifier toutes les requêtes suivantes.

2.4.2 Scanner de QR codes

- Affichage du flux caméra en temps réel via CameraX, avec analyse continue des images pour détecter un QR code (Google ML Kit).
- Dès qu'un code est détecté, son contenu est envoyé au serveur via l'endpoint de vérification.
- Le serveur interprète le contenu du QR code selon plusieurs formats possibles : un identifiant de commande au format JSON (`{"orderId": ...}`), un code au format « #GET-xxxx », ou un identifiant numérique brut.
- Affichage de l'événement actif (le plus proche dans le temps) en cours de contrôle, récupéré depuis la base partagée.
- Affichage de statistiques en direct : nombre de billets validés (utilisés), nombre de billets restants (validés mais non encore scannés) et nombre de refus.

2.4.3 Vérification et validation d'un billet

La logique de vérification, au cœur du backend, applique les règles suivantes selon le statut de la commande associée au QR code scanné :

- Commande au statut **VALIDATED** : le billet est accepté, son statut est immédiatement mis à jour à **USED** (utilisé), et un enregistrement « valide » est ajouté à l'historique. Les informations du porteur (nom, type d'utilisateur, mention, année, code et type de ticket) sont retournées à l'application.
- Commande au statut **USED** : le billet a déjà été scanné précédemment ; le système détecte le doublon, indique la date du premier scan et enregistre la tentative comme « doublon » dans l'historique.
- Commande au statut **PENDING** : le paiement n'a pas encore été validé par un administrateur GET Events ; le billet est refusé avec le motif « Paiement en attente de validation ».
- Commande au statut **CANCELLED** : le billet a été annulé ; il est refusé avec le motif « Ticket annulé par l'administrateur ».
- QR code illisible ou commande introuvable : le billet est refusé avec le motif « QR code non reconnu » ou « commande introuvable », et l'événement est tout de même enregistré dans l'historique pour traçabilité.

2.4.4 Affichage du résultat

- Écran « Ticket valide » : confirmation visuelle (vert) avec les informations du porteur du billet, puis retour automatique au scanner après 3,5 secondes pour fluidifier le contrôle des entrées.
- Écran « Ticket refusé » : alerte visuelle (rouge) précisant le motif exact du refus (déjà utilisé, annulé, paiement en attente, introuvable, etc.).

2.4.5 Historique des scans

- Liste de tous les scans effectués, triés du plus récent au plus ancien, avec le nom du porteur, sa catégorie, l'heure du scan et le statut (valide / refusé / doublon).

- Recherche par nom de client pour retrouver rapidement un scan particulier.
- Statistiques agrégées de l'historique : nombre total de billets validés, refusés et de doublons détectés.

2.5 Modèle de données

GET Ticket s'appuie sur le schéma existant de GET Events et y ajoute une seule nouvelle table :

Table	Description
Users (partagée)	Comptes utilisateurs de GET Events. GET Ticket y vérifie l'identité et le rôle (ADMIN) de l'agent connecté, ainsi que les informations du porteur d'un billet (nom, type, mention, année).
Orders (partagée)	Commandes de billets de GET Events. GET Ticket lit le statut d'une commande pour la vérification, et le met à jour (VALIDATED → USED) lors d'une validation réussie.
ScanHistory (nouvelle, créée par GET Ticket)	Historique de tous les scans effectués : identifiant de commande (le cas échéant), nom et catégorie du client, statut du scan (« valide », « refuse » ou « doublon »), motif de refus éventuel, et horodatage du scan.

Cette approche permet à GET Ticket de fonctionner comme une extension de GET Events plutôt que comme un système isolé : aucune donnée n'est dupliquée et toute action de contrôle (validation d'un billet) se reflète immédiatement dans GET Events, par exemple dans l'historique des commandes de l'utilisateur concerné.

2.6 API REST exposée par le backend

Le backend GET Ticket expose une API REST au format JSON, distincte de celle de GET Events (port 8081) mais s'appuyant sur la même base de données. Toutes les routes, à l'exception de la connexion, nécessitent un jeton JWT valide.

Route	Authentification	Description
POST /api/connexion	Aucune	Connexion d'un agent (compte ADMIN GET Events) et émission d'un jeton JWT.
GET /api/evenement-actif	JWT	Retourne l'événement actif (le plus proche dans le temps) en cours de contrôle.
GET /api/stats	JWT	Statistiques globales : billets validés (USED), restants (VALIDATED) et refusés.
GET /api/verifier/{codeQR}	JWT	Vérifie la validité d'un code QR donné en paramètre d'URL.
POST /api/verifier	JWT	Vérifie la validité d'un code QR transmis dans le corps de la requête.
POST /api/scan	JWT	Endpoint principal utilisé par le scanner : vérifie le code, met à jour le statut de la commande et enregistre le scan dans l'historique.
GET /api/historique	JWT	Retourne l'historique des scans (avec recherche optionnelle par nom) et les statistiques associées.

2.7 Sécurité

- Authentification par jeton JWT (signature HMAC256), partageant le même secret applicatif que GET Events afin d'assurer la compatibilité des comptes.
- Accès restreint aux seuls comptes possédant le rôle ADMIN : toute tentative de connexion d'un compte étudiant standard est rejetée (HTTP 403).
- Vérification des mots de passe via bcrypt, sans jamais transmettre ni stocker le mot de passe en clair.

- Toutes les routes fonctionnelles (scanner, statistiques, historique, événement actif) sont protégées par le plugin d'authentification JWT de Ktor et inaccessibles sans jeton valide.
- Idempotence du contrôle : un billet déjà scanné (statut USED) ne peut pas être validé une seconde fois, ce qui empêche la réutilisation frauduleuse d'un même billet.
- Gestion centralisée des erreurs via Status Pages, renvoyant des messages d'erreur génériques au client en cas d'exception interne.